

1. Product Thinking

Dominik Kundel's Astra field report describes a change in how much detail he needed to prescribe. Given a rough idea and relevant reference conversations, Astra inferred expected image-editor interactions, familiar shortcuts, layers, and a feedback interface with comments, priorities, editing, and resolution state.

This suggests a capability: the model can fill in product structure from conventions and evidence. It does not establish that the inferred structure is always desirable.

2. The Inference Argument

1. Many product decisions are constrained by established conventions and nearby evidence.
2. A model that can inspect those constraints may infer reasonable missing details.
3. Writing every inferred detail by hand can be costly and can freeze weak assumptions before exploration.
4. Therefore a capable agent should sometimes be given the outcome and references rather than a complete interaction script.

The conclusion is conditional. It applies where conventions are acceptable and errors are cheap to review.

3. The Prescription Objection

If the product deliberately breaks convention, contains brand-critical interactions, or carries legal/accessibility requirements, leaving the detail implicit is dangerous. A fluent model may choose the common pattern precisely when the uncommon pattern is the point.

Kundel's own advice contains the answer: if narrow behavior matters, be prescriptive. The skill is deciding where novelty or constraint lives.

4. Reference Quality

A good reference reduces the space of reasonable interpretations. Three screenshots from the same system with coherent spacing and interaction can teach more than a paragraph of adjectives. A real sibling component can establish API and naming conventions. A user-flow recording can expose timing and state changes absent from a static design.

Weak references create imitation without structure. A famous brand homepage may have no bearing on a developer tool's dense table.

5. Apps as Cognitive Extensions

Kundel's LEGO and Blender examples are notable because Astra did not stay inside a code-only workflow. It used code to construct or transform data, then opened the domain application to inspect and iterate. The application becomes part of the reasoning environment: it reveals constraints and feedback unavailable in plain text.

This supports a general rule: give the agent the same class of instrument a competent human would use when that instrument exposes relevant state.

6. Retry Abandoned Tasks

Kundel recommends revisiting tasks that prior models could not do. This is sensible after a capability shift, but a fair retry should preserve the task's success criteria. Otherwise the user may unconsciously lower the bar because the new output is impressive.

Record the old failure if available. Define the current target before the new run. Then compare the new result against the requirement, not against memory of frustration.

7. Reasoning Level

His other practical recommendation is to begin Astra at Low or Medium rather than assuming Max. The deeper principle is empirical calibration. Model generation changed, so inherited settings should be revalidated. A workflow optimized around Sol may be overbuilt for Astra in both reasoning effort and skill scaffolding.

8. Verification Contract

The final Kundel recommendation is the most general: tell Astra what finished means, what to verify, and when to hand work back. Product thinking without a verification contract can expand scope. A clear end state lets initiative operate inside a boundary.

References: Dominik Kundel, "Five things I learned from using Astra"; OpenAI, GPT-6 Astra launch; OpenAI, GPT-6 Astra model guidance.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 19: Kundel on Product Thinking

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>